COURSE CODE: 17YCM603

COURSE NAME: SEARCH ALGORITHMS

UNIT-3 SYLLABUS

LECTURE: 3HRS/WEEK

MARKS: 100

OBJECTIVES:

To Learn the basics of search algorithms.
To understand and apply various types of search algorithms.
To understand the concept of Heuristic Search.
To apply different types of Hill Climbing Algorithms.
To learn the various crypto currency and bitcoin algorithm

OUTCOMES

At the end of the course, the learners will be able to:	
1	Implement search algorithms in real world applications.
2	Apply heuristic search as a searching technique.
3	Apply hill climbing algorithm to solve the problems
4	Explain and implement BFS and DFS algorithm
5	Discuss cryptocurrency, bitcoin and its algorithm.

TEXTBOOKS

Text Books
1. Thomas H. Cormen , Charles E. Leiserson , Ronald L. Rivest , Clifford Stein, “Introduction to Algorithms, 3rd Edition, The MIT Press, 3rd Edition, 2009. 2. Kosko, B, “Neural Networks and FuzzySystems: A Dynamical Approach to Machine Intelligence”, PrenticeHall, NewDelhi, 2004.
Reference Books
1.Jack M. Zurada, “Introduction to Artificial Neural Systems”, PWS Publishing Co., Boston, 2002. 2. Klir G.J. & Folger T.A., “Fuzzysets, Uncertainty and Information”, Prentice–Hall of India Pvt. Ltd., New Delhi, 2008

SYLLABUS:

Module 1: Heuristic Search: introduction to Heuristic Search, choosing a good Heuristic, The 8 Puzzle Problem, Monotonicity Modified Travelling Salesman Problem,

Module 2: Best Fit Search, A* Algorithm, Iterative Deepening A* Search, Generalization of Problems

MODULE 1

HEURISTIC SEARCH

INTRODUCTION OF HEURISTIC SEARCH

Heuristics operates on the search space of a problem to find the best or closest-to-optimal solution via the use of systematic algorithms. In contrast to a brute-force approach, which checks all possible solutions exhaustively, a heuristic search method uses heuristic information to define a route that seems more plausible than the rest. Heuristics, in this case, refer to a set of criteria or rules of thumb that offer an estimate of a firm's profitability. Utilizing heuristic guiding, the algorithms determine the balance between exploration and exploitation, and thus they can successfully tackle demanding issues. Therefore, they enable an efficient solution finding process.

SIGNIFICANCE OF HEURISTIC SEARCH IN AI

The primary benefit of using heuristic search techniques in AI is their ability to handle large search spaces. Heuristics help to prioritize which paths are most likely to lead to a solution, significantly reducing the number of paths that must be explored. This not only speeds up the search process but also makes it feasible to solve problems that are otherwise too complex to handle with exact algorithms.

COMPONENTS OF HEURISTIC SEARCH

Heuristic search algorithms typically comprise several essential components:

State Space: This implies that the totality of all possible states or settings, which is considered to be the solution for the given problem.

Initial State: The instance in the search tree of the highest level with no null values, serving as the initial state of the problem at hand.

Goal Test: The exploration phase ensures whether the present state is a terminal or consenting state in which the problem is solved.

Successor Function: This create a situation where individual states supplant the current state which represent the possible moves or solutions in the problem space.

Heuristic Function: The function of a heuristic is to estimate the value or distance from a given state to the target state. It helps to focus the process on regions or states that has prospect of achieving the goal.

APPLICATIONS OF HEURISTIC SEARCH

Heuristic search techniques find application in a wide range of problem-solving scenarios, including:

1. **Pathfinding:** Discovery, of the shortest distance that can be found from the start point to the destination at the point of coordinates or graph.

2. **Optimization:** Solving the problem of the optimal distribution of resources, planning or posting to achieve maximum results.

3. **Game Playing:** The agency of AI with some board games, e.g., chess or Go, is on giving guidance and making strategy-based decisions to the agents.

4. **Robotics:** Scheduling robots` location and movement to guide carefully expeditions and perform given tasks with high efficiency.

5. **Natural Language Processing:** Language processing tasks involving search algorithms, such as parsing or semantic analysis, should be outlined. That means.

ADVANTAGES OF HEURISTIC SEARCH TECHNIQUES

Heuristic search techniques offer several advantages:

Efficiency: As they are capable of aggressively digesting large areas for the more promising lines, they can allot more time and resources to investigate the area.

Optimality: If the methods that an algorithm uses are admissible, A* guarantees of an optimal result.

Versatility: Heuristic search methods encompass a spectrum of problems that are applied to various domains of problems.

LIMITATIONS OF HEURISTIC SEARCH TECHNIQUES

Heuristic Quality: The power of heuristic search strongly depends on the quality of function the heuristic horizon. If the heuristics are constructed thoughtlessly, then their level of performance may be low or inefficient.

Space Complexity: The main requirement for some heuristic search algorithms could be a huge memory size in comparison with the others, especially in cases where the search space considerably increases.

Domain-Specificity: It is often the case that devising efficient heuristics depends on the specifics of the domain, a challenging obstruction to development of generic approaches.

THE REASONS FOR GOOD HEURISTIC SEARCH

A good heuristic search is characterized by its ability to efficiently find solutions to complex problems, often in less time and with fewer resources than other methods. Here are the main reasons why heuristic search can be particularly effective:

1. **Informed Decision-Making**

Guidance: Heuristics provide guidance on which paths are more likely to lead to a goal, reducing the need to explore every possible option.

Prioritization: By evaluating potential paths based on heuristics, the search can prioritize those that appear most promising.

2. **Efficiency**

Reduced Search Space: Heuristics help prune the search space, focusing only on the most relevant parts of the problem, which saves time and computational resources.

Speed: By avoiding unnecessary explorations, heuristic searches can arrive at solutions faster than uninformed search methods.

3. **Problem-Specific Knowledge**

Domain Expertise: A well-designed heuristic incorporates domain-specific knowledge that can dramatically improve search performance by leveraging insights specific to the problem.

4. **Optimality (in some cases)**

Admissible Heuristics: When heuristics are admissible and consistent, algorithms like A* can guarantee finding the optimal solution.

Balance: Good heuristics strike a balance between being informative enough to guide the search effectively and simple enough to compute quickly.

5. **Flexibility**

Adaptability: Heuristic searches can be tailored to different problems by designing appropriate heuristics, making them versatile tools across various domains.

Scalability: They can be adapted to handle larger problem instances by adjusting the heuristic or search parameters.

6. **Resource Management**

Memory Usage: Techniques like beam search use heuristics to limit memory usage by restricting the number of nodes kept in memory at any time.

Computational Resources: Efficient use of computational resources is achieved by focusing efforts on promising paths rather than exhaustive exploration.

7. **Handling Complexity**

Complex Problem Solving: Heuristic search is particularly useful in solving problems that are too complex for brute-force methods, such as NP-hard problems.

Approximation: Even when optimal solutions are not feasible, heuristic search can provide good approximations that are sufficient for practical purposes.

8 Puzzle Problem

The 8-puzzle is a well-known problem in the field of artificial intelligence and puzzle-solving. It serves as an intriguing model problem with various applications and is widely used to explore heuristic search and state-space exploration.

WHAT IS 8 PUZZLE PROBLEM IN AI AND ITS RELEVANCE

The 8-puzzle, often regarded as a small, solvable piece of a larger puzzle, holds a central place in AI because of its relevance in understanding and developing algorithms for more complex problems.

RULES AND CONSTRAINTS FOR 8 PUZZLE PROBLEM

To tackle the 8-puzzle, it's crucial to comprehend its rules and constraints:

- The 8-puzzle is typically played on a 3x3 grid, which provides a 3x3 square arrangement for tiles. This grid structure is fundamental to the problem's organization.
- The puzzle comprises 8 numbered tiles (usually from 1 to 8) and one blank tile. These numbered tiles can be slid into adjacent positions (horizontally or vertically) when there's an available space, which is occupied by the blank tile.
- The objective of the 8-puzzle is to transform an initial state, defined by the arrangement of the tiles on the grid, into a specified goal state. The goal state is often a predefined configuration, such as having the tiles arranged in ascending order from left to right and top to bottom, with the blank tile in the bottom-right corner.

FUNDAMENTAL STATES OF 8 PUZZLE PROBLEM

Initial and Goal States:

In the context of the 8-puzzle, two fundamental states are of particular importance: the initial state and the goal state.

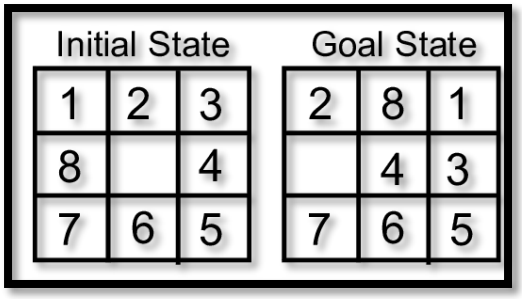
1. Initial State:

- The initial state of the 8-puzzle represents the starting configuration. It's the state from which the puzzle-solving process begins.
- The initial state can be any arrangement of the tiles, which can be specified manually or generated randomly.
- The problem-solving algorithm aims to transform the initial state into the goal state using a sequence of valid moves.

2. Goal State:

- The goal state represents the desired configuration that the puzzle should reach.

- In most cases, the goal state involves arranging the numbered tiles in ascending order from left to right and top to bottom, with the blank tile in the bottom-right corner.
- Achieving the goal state demonstrates the successful solution of the puzzle.



HOW AI TECHNIQUE IS USED TO SOLVE THE 8 PUZZLE PROBLEM

Choice of Algorithms:

When approaching the eight puzzle problem in ai, the choice of search algorithm is critical. Several algorithms can be used, each with its advantages and trade-offs. Here, we discuss three common choices:

1. Breadth-First Search (BFS):

- BFS is an uninformed search algorithm that explores states level by level.
- It guarantees finding the shortest path to the goal state, making it a reliable choice for the 8-puzzle problem.
- However, it may consume substantial memory, as it stores all explored states.

2. Depth-First Search (DFS):

- DFS is another uninformed search algorithm that explores states deeply before backtracking.
- It is memory-efficient but does not guarantee finding the optimal solution.
- DFS can be suitable for solving the 8-puzzle when memory constraints are a concern.

3. A* Algorithm:

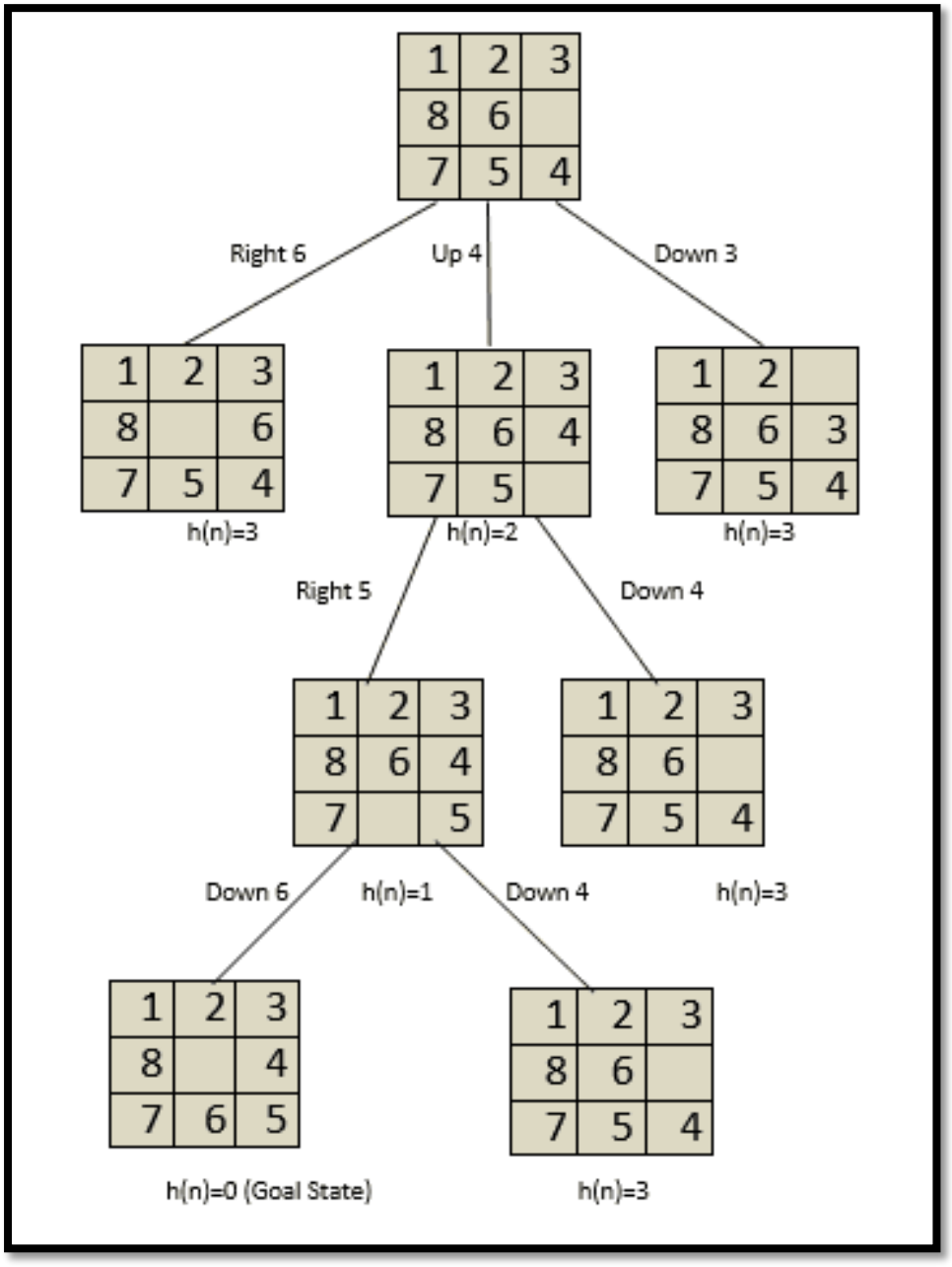
- A* is an informed search algorithm that combines the cost to reach a state (g-value) with a heuristic estimate of the cost to reach the goal (h-value).
- It is widely used for solving problems like the 8-puzzle, as it finds optimal solutions efficiently.
- A* relies on an admissible heuristic to guide the search.

INTRODUCING HEURISTIC FUNCTIONS:

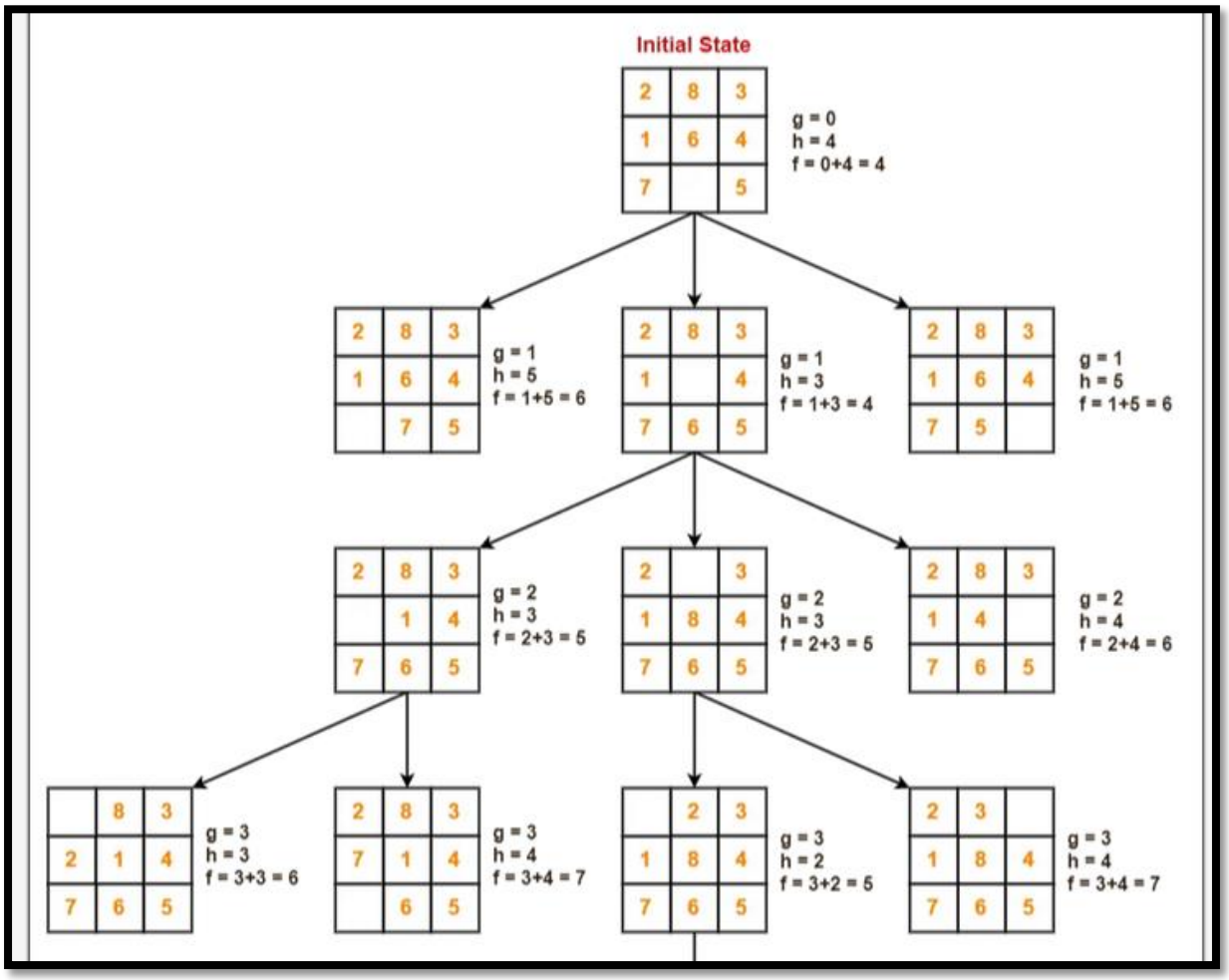
Heuristic functions are a vital component of informed search algorithms, like A*. They provide estimates of the cost to reach a goal state from a given state. Here's why they are essential:

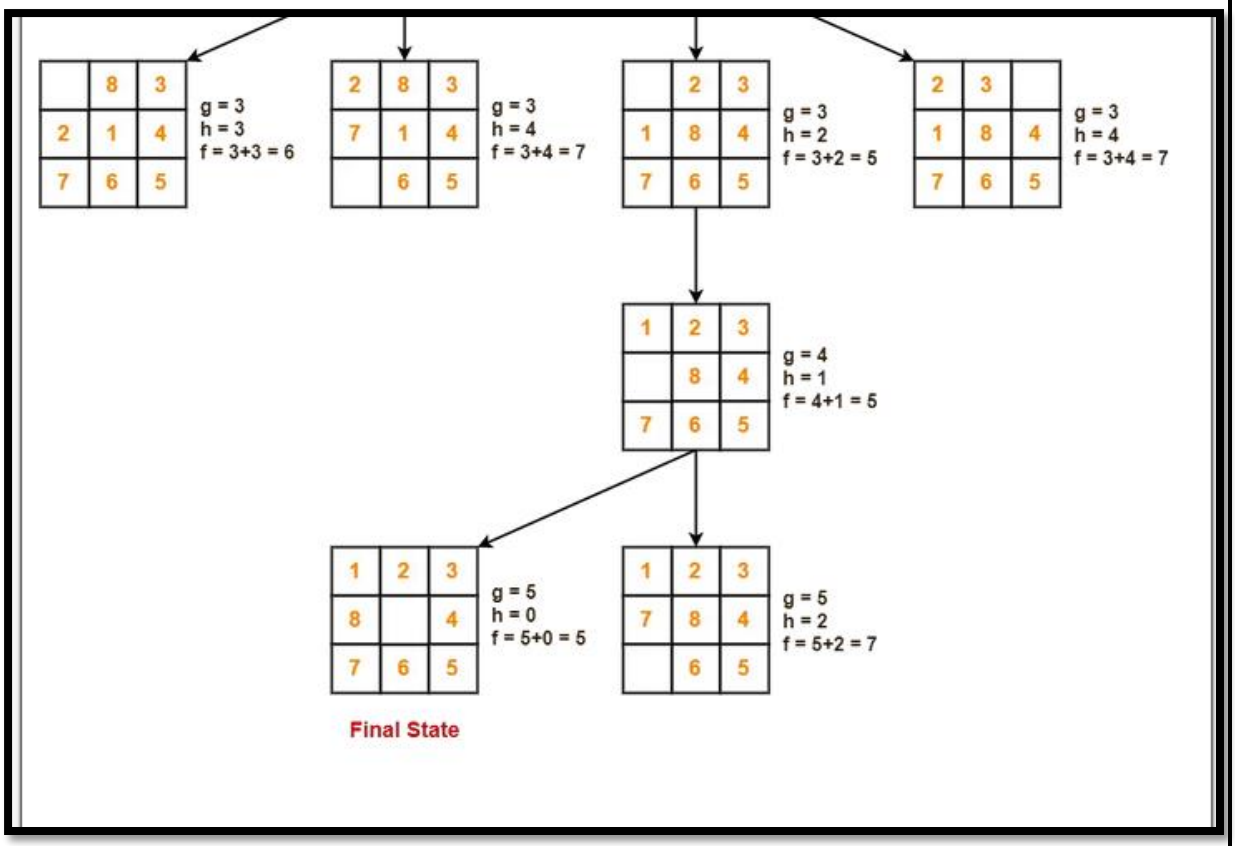
- **Importance of Heuristic Functions:** Heuristic functions are used to guide search algorithms by providing a measure of how promising a state is in reaching the goal. In other words, they help the algorithm make informed choices about which states to explore next.
- **Informed vs. Uninformed Search:** Informed search algorithms, like A*, use heuristic functions to focus on more promising states, making them significantly more efficient than uninformed algorithms.

EXAMPLE 1



EXAMPLE 2


Initial State
2 8 3
1 6 4
7 5
g = 0
h = 4
f = 0+4 = 4
2 8 3
1 6 4
7 5
g = 1
h = 5
f = 1+5 = 6
2 8 3
1 4
7 6 5
g = 1
h = 3
f = 1+3 = 4
2 8 3
1 6 4
7 5
g = 1
h = 5
f = 1+5 = 6
2 8 3
1 4
7 6 5
g = 2
h = 3
f = 2+3 = 5
2 3
1 8 4
7 6 5
g = 2
h = 3
f = 2+3 = 5
2 8 3
1 4
7 6 5
g = 2
h = 4
f = 2+4 = 6
8 3
2 1 4
7 6 5
g = 3
h = 3
f = 3+3 = 6
2 8 3
7 1 4
6 5
g = 3
h = 4
f = 3+4 = 7
2 3
1 8 4
7 6 5
g = 3
h = 2
f = 3+2 = 5
2 3
1 8 4
7 6 5
g = 3
h = 4
f = 3+4 = 7



PSEUDOCODE FOR 8 PUZZLE PROBLEM

```
function a_star_search(start_state, goal_state):
    open_list = PriorityQueue()
    open_list.put(start_state, priority=0)
    came_from = {}
    cost_so_far = {}
    came_from[start_state] = None
    cost_so_far[start_state] = 0

    while not open_list.empty():
        current = open_list.get()

        if current == goal_state:
            return reconstruct_path(came_from, current)
```

```
        for neighbor in get_neighbors(current):
            new_cost = cost_so_far[current] + 1  # g(n)
            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                priority = new_cost + heuristic(neighbor, goal_state)  # f(n) = g + h
                open_list.put(neighbor, priority)
                came_from[neighbor] = current

    return "No solution found"

function get_neighbors(state):
    # Return a list of valid states after sliding a tile
    moves = []
    find position of 0 in the state
    for each valid move (up/down/left/right):
        generate new state by swapping 0 with the tile in that direction
        add new state to moves
    return moves

function heuristic(state, goal_state):
    # Manhattan Distance
    total_distance = 0
    for tile in 1 to 8:
        current_position = find tile in state
        goal_position = find tile in goal_state
        distance = abs(current.x - goal.x) + abs(current.y - goal.y)
        total_distance += distance
    return total_distance
```

```
function reconstruct_path(came_from, current):
    path = []
    while current is not None:
        path.append(current)
        current = came_from[current]
    path.reverse()
    return path
```

ADVANTAGES OF 8 PUZZLE PROBLEM

- **Educational Value:**

Great for understanding search algorithms (BFS, DFS, A*, UCS). Demonstrates heuristic design and pathfinding in a simple domain.

- **Well-defined Problem:**

Clearly defined states, actions, goal state, and rules. Easy to represent in code using arrays or tuples.

- **Small State Space:**

Only 9! = 362,880 possible states, making it tractable for most search algorithms with heuristics.

- **Supports Heuristic Search:**

Ideal for testing heuristics like Manhattan distance or misplaced tiles in A*.

- **Benchmark for AI Algorithms:**

Frequently used as a standard problem to compare performance and efficiency of AI strategies.

- **No Randomness:**

Fully deterministic — the same inputs always lead to the same outputs, simplifying testing and evaluation.

DISADVANTAGES OF 8 PUZZLE PROBLEM

- **Limited Real-World Application**:

Though good for learning, it's abstract and doesn't directly map to many real-world problems.

- **Slow with Uninformed Search:**

Without heuristics, BFS and DFS can be inefficient and slow.

- **Not All States Are Solvable:**

Only half of the possible arrangements are solvable; needs a check for solvability.

- **Scales Poorly:**

As you increase to 15-puzzle or 24-puzzle, complexity and computation time rise exponentially.

- **Memory Usage:**

Search trees (especially with BFS or UCS) can consume a lot of memory if not optimized.

- **Difficult for Beginners:**

Implementing efficient heuristics and handling state management (like hashing and comparing states) can be tricky for new learners.

MONOTONICITY MODIFIED TRAVELLING SALESMAN PROBLEM

TRAVELLING SALESMAN PROBLEM

The **Travelling Salesman Problem (TSP)** and the **Monotonicity Modified Travelling Salesman Problem (MMTSP)** differ primarily in their constraints and objectives. Here's a breakdown of the differences:

---

1. Travelling Salesman Problem (TSP):

- **Objective**: The goal is to find the shortest possible route that allows the salesman to visit each city exactly once and return to the starting city.
- **Constraints**:
- Every city must be visited exactly once.
- The total travel distance or cost must be minimized.
- **General Nature**: TSP assumes no restrictions on the order of city visits; any permutation of the cities can potentially be a valid solution if it minimizes the cost.
- **Applications**:
- Logistics and delivery optimization.
- Circuit design.
- Route planning.

---

2. Monotonicity Modified Travelling Salesman Problem (MMTSP):

- **Objective**: In addition to minimizing the distance or cost, MMTSP introduces an extra constraint that restricts the sequence of visits based on monotonicity rules.
- **Monotonicity Constraint**:
- The order in which cities are visited must follow certain monotonic patterns, such as non-decreasing or non-increasing values in one dimension (e.g., x-coordinates, y-coordinates, or time).
- For example, if cities are plotted on a coordinate system, the salesman may only be allowed to move in a way that follows increasing x-values or y-values.
- **Effect of Modification**:
- This constraint reduces the number of feasible routes compared to TSP since not all permutations of cities are valid.
- It can simplify the problem in specific contexts but can also make the solution space harder to navigate due to imposed restrictions.
- **Applications**:
- Specific cases in network routing where monotonic constraints apply.
- Problems involving sequential ordering or data dependency.

Key Differences:

Feature	TSP	MMTSP
Order of Visits	No restrictions, any order is valid.	Monotonic constraints restrict order.
Solution Space	Larger; all permutations are allowed.	Smaller due to monotonic restrictions.
Complexity	NP-hard problem.	NP-hard, possibly harder due to added constraints.
Applicability	General optimization problems.	Problems with spatial or temporal monotonicity requirements.

Definition of MMTSP:

In MMTSP, the goal is to find the shortest possible route for visiting a set of cities (or nodes) exactly once and returning to the starting city. However, unlike standard TSP, the order in which cities are visited must follow a **monotonic constraint**.

This monotonicity can be defined based on:

1. **Spatial coordinates** (e.g., x-coordinates or y-coordinates).
2. **Temporal order** (e.g., increasing or decreasing time).
3. **Any other parameter** where monotonicity is meaningful.

Key Features of MMTSP:

1. **Monotonic Constraint**:

- The salesman must visit cities in a sequence that respects monotonicity rules. For example:
- If cities are represented by their coordinates (x, y), the salesman might only be allowed to move in a pattern where the x-coordinates are non-decreasing.
- Alternatively, the y-coordinates might need to follow a non-increasing order.
- This eliminates certain routes that would be valid in a standard TSP but violate monotonicity.

2. **Objective**:

- Minimize the total travel cost/distance while adhering to the monotonicity constraint.

3. **Solution Space**:

- The monotonic constraint reduces the number of feasible routes compared to standard TSP.

- This can make the optimization problem easier or harder depending on the setup and the nature of the constraints.

Mathematical Representation:

Let $G(V,E)$ be a graph where:

- V is the set of cities/nodes.
- E is the set of edges representing distances between nodes.

In MMTSP:

1. The salesman must find a Hamiltonian cycle that minimizes travel distance.
2. The sequence of visited nodes $v1,v2,v3,\ldots,vn$ must satisfy monotonicity conditions:
- For spatial monotonicity: $x(vi) \leq x(vi+1)$ or $y(vi) \geq y(vi+1)$, etc.
- For temporal monotonicity: $t(vi) \leq t(vi+1)$.

Applications:

MMTSP is useful in scenarios where monotonic behavior is required due to physical constraints or logical ordering:

- **Robotics**: Efficient path planning for robots in constrained environments.
- **Logistics**: Delivery routes constrained by geographic monotonicity (e.g., traveling northward only).
- **Data sequencing**: Problems involving sequential dependency or ordering constraints.

Challenges:

1. **Reduction in Feasible Routes**:

- The monotonicity constraint limits the flexibility of route planning, making certain solutions invalid even if they have shorter distances.
2. **Computational Complexity**:

- While still NP-hard like standard TSP, MMTSP introduces additional complexity due to constraints, requiring specialized algorithms for solving.

Key Features of Monotonicity Modified Travelling Salesman Problem (MMTSP):

1. **Monotonic Constraint**:

- The sequence in which cities or nodes are visited must adhere to monotonicity rules. These rules can be based on:
- **Spatial coordinates**: The salesman may only move in a direction where the x-coordinates are non-decreasing (or non-increasing), or the y-coordinates follow a similar pattern.

- **Temporal order**: The visits must occur in a sequence where time increases or decreases.
- **Other attributes**: Any parameter that requires monotonic behavior (e.g., distance, priority level).

2. **Reduced Feasible Solution Space**:

- Unlike the classic Travelling Salesman Problem (TSP), not all possible permutations of cities are valid solutions. The monotonicity constraint eliminates routes that don't comply with the defined monotonic pattern, reducing the set of feasible routes.

3. **Objective**:

- Similar to TSP, the main goal is to minimize the total travel cost or distance while fulfilling monotonicity requirements.

4. **Hamiltonian Cycle with Restrictions**:

- A valid solution still requires a Hamiltonian cycle (a route that visits each city exactly once and returns to the starting point). However, this cycle must respect monotonic constraints, adding complexity to the problem.

5. **Complexity**:

- While MMTSP is NP-hard like standard TSP, the monotonicity constraint can increase computational complexity by requiring more specific algorithms or heuristics. Some routes may become infeasible due to strict constraints.

6. **Applications**:

- MMTSP is particularly useful where monotonic behavior is required due to physical or logical constraints:
- **Geographic Routing**: Planning routes constrained by geographic monotonicity (e.g., traveling eastward or northward).
- **Data Sequencing**: Problems that involve sequential dependency.
- **Robotics Path Planning**: Robots may need to navigate in a monotonic fashion due to physical constraints or safety requirements.
- **Delivery Scheduling**: Routes constrained by time windows or geographic order.

7. **Trade-Offs**:

- The monotonic constraint simplifies certain aspects (e.g., fewer permutations to evaluate compared to TSP), but it also limits flexibility, potentially leading to suboptimal solutions when compared to unconstrained TSP.

8. **Algorithm Adaptation**:

- Solving MMTSP often requires adapting existing TSP algorithms or developing new heuristics to account for monotonic constraints. Examples include dynamic programming, branch-and-bound techniques, and monotonic-specific greedy algorithms.

Advantages of Monotonicity Modified Travelling Salesman Problem (MMTSP):

1. **Simplifies Route Planning in Specific Scenarios**:

- By enforcing monotonicity constraints (e.g., non-decreasing x-coordinates or y-coordinates), the problem may become easier to solve in cases where monotonic movement aligns naturally with the environment or task requirements.
- This is especially relevant in constrained physical or logical scenarios like geographic routing or sequential workflows.

2. **Reduced Solution Space**:

- The monotonicity constraint eliminates invalid routes, reducing the number of possible permutations to evaluate. This can simplify computations and improve efficiency in scenarios where monotonicity is essential.

3. **Improved Applicability to Real-World Constraints**:

- Many real-world problems inherently require monotonic behavior, such as:
- **Robotics Path Planning**: Robots navigating in a structured environment may need to follow monotonic paths for safety or mechanical reasons.
- **Delivery Routing**: Geographically constrained delivery systems (e.g., traveling along streets in a single direction) are better modeled using MMTSP.
- **Data Sequencing**: Tasks requiring sequential ordering can benefit from monotonic constraints.

4. **Better Alignment with Physical or Logical Laws**:

- Monotonicity captures physical or logical restrictions that arise due to real-world limitations (e.g., geographical restrictions, directional constraints, or time dependencies). This makes MMTSP solutions more practical and implementable compared to unconstrained TSP.

5. **Prevents Overlapping or Backtracking**:

- Monotonic constraints can help avoid overlapping or backtracking routes, which might be inefficient or impractical in certain applications (e.g., traveling in one direction on a highway or navigating through ordered data).

6. **Facilitates Optimization for Specialized Scenarios**:

- In cases where monotonic constraints are required, MMTSP provides a framework that tailors optimization specifically to those needs, ensuring that the solution is both feasible and efficient.

7. **Streamlined Algorithm Design for Specific Use Cases**:

- The reduced solution space and structured constraints allow for specialized algorithms (e.g., dynamic programming or greedy methods) that exploit monotonicity properties for faster computation.

8. **Applicability to Sequential Data Problems**:

- Beyond geographic or spatial routing, MMTSP applies well to problems involving data sequencing or dependency-based workflows where monotonic order is required.

Real-World Example Benefits:

- **Robotics**: A warehouse robot tasked with picking items may need to follow a monotonic path to avoid collisions or simplify movement patterns.
- **Logistics**: Delivery systems constrained by traffic rules (e.g., one-way roads or directional flow) align naturally with monotonic constraints.
- **Scheduling**: Tasks requiring sequential ordering based on time, priority, or other attributes directly benefit from the monotonic structure.

MODULE 2

BEST FIT SEARCH

DEFINE BEST FIT SEARCH

Best Fit Search is a heuristic search strategy used in optimization problems where the goal is to find the most suitable (best-fitting) solution from a set of possible candidates based on a given criterion. Unlike uninformed search algorithms (e.g., BFS, DFS), Best Fit Search uses a **scoring or fitness function** to evaluate and prioritize options, similar to a **greedy approach** but with a focus on optimal matching.

HOW BEST FIT SEARCH WORKS?

1. **Define a Fitness Function:**
   - A function that evaluates how well a candidate solution meets the desired criteria (e.g., minimizing cost, maximizing efficiency, closest match).
   - Example: In memory allocation, fitness could be the smallest block that fits a process.
2. **Evaluate All Candidates:**
   - The algorithm examines all possible options and computes their fitness scores.
3. **Select the Best Option:**
   - The candidate with the highest (or lowest, depending on the problem) fitness score is chosen.

WORKING MECHANISM OF BEST FIT SEARCH

Best-Fit Search follows these steps:

1. Initialize a priority queue with the start node.
2. Sort nodes by their heuristic values h(n).
3. Expand the node with the lowest heuristic value (best estimated path to the goal).
4. Add neighboring nodes to the priority queue.
5. Repeat until the goal is reached or all nodes are explored.

APPLICATIONS OF BEST FIT SEARCH

1. Memory Management (OS dynamic allocation)
2. Bin Packing Problems (optimizing container usage)
3. Job Scheduling (assigning tasks to the most suitable machine)
4. Database Query Optimization (selecting the best execution plan)

ADVANTAGES AND DISADVANTAGES OF BEST FIT SEARCH

ADVANTAGES

1. Fast Execution: Greedy approach makes it efficient.
2. Simple Implementation: Uses only a heuristic function.
3. Works well for certain optimization problems like scheduling & bin packing.

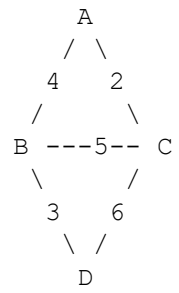
DISADVANTAGES

1. Not Guaranteed to Find the Best Path: May get stuck in local optimal
2. Does Not Consider Path Cost: Only looks at heuristic values, ignoring actual distances.
3. Incomplete in Some Cases: If a node with a low heuristic leads to a dead-end, the algorithm may fail.

EXAMPLE OF BEST FIT SEARCH

Consider a graph where we need to find the shortest path from **A to D**.

Graph Representation



Heuristic Function h(n)

Node	Heuristic h(n) (Estimated Distance to Goal)
A	7
B	6
C	2
D	0 (Goal)

Algorithm Execution

Start at A

h(A)=7h(A) = 7h(A)=7

Neighbors: B (h=6), C (h=2)

Choose C (smallest heuristic).

Move to C

h(C)=2h(C) = 2h(C)=2

Neighbors: A (7), B (6), D (0)

Choose D (goal reached).

✅ Final Path: A → C → D (Total Cost: 2 + 6 = 8)

The shortest path A → B → D (cost 7) is ignored!

PYTHON IMPLEMENTATION

Python Code for Best-Fit Search

```
import heapq

def best_fit_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue (min-heap)
    heapq.heappush(open_list, (heuristic[start], start))
# Add start node
    visited = set()

    while open_list:
        _, current = heapq.heappop(open_list)  # Get node
with best heuristic

        if current == goal:
            print("Goal reached:", goal)
            return

        visited.add(current)

        for neighbor, _ in graph[current]:  # Explore
neighbors
            if neighbor not in visited:
                heapq.heappush(open_list,
(heuristic[neighbor], neighbor))

    print("Goal not found!")

# Example Graph (Adjacency List)
graph = {
```

```
    'A': [('B', 4), ('C', 2)],
    'B': [('A', 4), ('D', 3), ('C', 5)],
    'C': [('A', 2), ('B', 5), ('D', 6)],
    'D': []
}

# Heuristic Values
heuristic = {
    'A': 7, 'B': 6, 'C': 2, 'D': 0  # Estimated cost to
reach goal (D)
}

# Run Best-Fit Search
best_fit_search(graph, heuristic, 'A', 'D')
```

A*(star) SEARCH

DEFINE A* SEARCH

A* search algorithm is a path finding algorithm that finds the single-pair shortest path between the start node (source) and the target node(destination) of a weighted graph. The algorithm not only considers the actual cost from the start node to the current node (g) but also tries to estimate the cost will take from the current node to the target node using heuristics (h). Then it selects the node that has the lowest f-value (f=g+h) to be the next node to move until it hits the target node. Dijkstra's algorithm is a special case of A* algorithm where heuristic is 0 for all nodes.

HOW A* SEARCH WORKS

The formula for A* algorithm considers two functions, **g(n) and h(n)**. Imagine that we are currently at node n, g(n) tells us the actual cost we took from the start node to the current node. h(n) is a heuristic function that estimates the cost we will take to get to the target node from the current node. Therefore, h(n) is an educated guess and it is crucial to determine the performance of A* algorithm. Based on the sum of the g(n) and h(n), f(n), the algorithm is able to estimate the total cost from the start node to the end node.

f(n) = g(n) + h(n)f(n): the estimate of the total cost from start node to target node through node n

g(n): actual cost from start node to node n

h(n): estimated cost from node n to target node

The algorithm selects the **minimum f-value node** as the next current node to explore and continuously update g(n) and h(n) if a better path is found for nodes it encountered. This process continues until the algorithm reaches the target node.

A* SEARCH VS. DIJIKSTRAS ALGORITHM

A* Algorithm only finds the shortest path between the start node and the target node, whereas Dijkstra's algorithm finds the shortest path from the start node to all other nodes because every node in a graph is a target node for it.

A* algorithm runs faster than Dijkstra's algorithm because it uses a heuristic to direct in the correct direction towards the target node. However, Dijkstra's algorithm expands evenly in all of the different available directions because it has no knowledge regarding the target node before hand, it always processes the node that is the closest to the start node based on the cost or distance it already took.

STEP BY STEP PSEUDOCODE

1. Initialize the Open and Closed Lists:
    - Open List contains nodes to be evaluated.
    - Closed List contains nodes that have already been evaluated.
2. Add the start node to the Open List.
3. Loop until the Open List is empty or the goal is reached:
    - Select the node from the Open List with the lowest f(n), where f(n) = g(n) + h(n).
    - If the current node is the goal, reconstruct the path and return it.
4. For the selected node:
    - Remove it from the Open List and add it to the Closed List.
    - Evaluate all neighboring nodes:
        - If a neighbor is in the Closed List, skip it.
        - If a neighbor is in the Open List, check if this path is better (lower g(n)); update if necessary.
        - Otherwise, calculate g(n), h(n), and f(n), then add the neighbor to the Open List.
5. End the loop once the goal is reached or the Open List is empty (no path found).
6. Return the reconstructed path or indicate that no valid path exists.

APPLICATIONS OF A* SEARCH

The *A* algorithm* is widely applied in various industries due to its ability to efficiently find the shortest path between two points while considering cost and distance. Below are some of the key real-world applications of A* across different fields:

1. GPS Navigation

One of the most well-known applications of A* is in GPS navigation systems. These systems rely on A* to calculate the shortest and most efficient route between two locations while accounting for various factors like traffic, road conditions, and travel time.

Example: In applications like Google Maps, A* helps determine the optimal driving or walking route, avoiding congested roads and reducing travel time. How it works: A* uses real-time data to update the path dynamically, recalculating as needed to adapt to changing traffic patterns. The algorithm ensures that users are always guided via the quickest available route to their destination. GPS navigation showcases the real-time capabilities of A*, highlighting its ability to adapt to new data and changing environments.

2. Video Games

In the video game industry, A* is extensively used for pathfinding and controlling AI characters. Game environments often consist of grid-based maps with obstacles, and A* is the algorithm of choice for determining the shortest path for in-game characters.

Strategy Games: Games like StarCraft and Age of Empires use A* to help AI-controlled units navigate through complex maps, avoiding obstacles and enemy units.

NPC Pathfinding: Non-player characters (NPCs) in open-world games also rely on A* to move efficiently and respond to player actions, such as navigating through buildings or avoiding hazards.

A* ensures that characters move realistically and efficiently, enhancing the player experience by providing smooth and intelligent navigation.

3. Robotics

Robotics is another field where A* plays a critical role, especially in navigation and motion planning for autonomous robots. Robots in various environments, such as warehouses or hospitals, need to navigate through spaces filled with obstacles, and A* helps them find the optimal path.

Autonomous Robots: Robots use A* to navigate through predefined environments by calculating the shortest route to their destination while avoiding dynamic obstacles.

Drones and UAVs: Drones and unmanned aerial vehicles (UAVs) apply A* for pathfinding, ensuring safe flight paths over complex terrains or populated areas.

A* is particularly useful in robotics because it allows robots to navigate both static and dynamic environments while optimizing travel paths, ensuring efficient and safe operations.

4. Network Routing

In network routing, A* helps optimize the flow of data across interconnected networks by calculating the most efficient route for data packets. This is crucial for minimizing latency and ensuring smooth communication between devices in a network.

Internet Traffic: A* finds the optimal path for data packets in large networks, ensuring that they are transmitted with minimal delays.

Optimizing Bandwidth Usage: In large-scale networks, A* can help distribute network traffic more evenly, preventing bottlenecks and improving overall performance.

By optimizing packet routing, A* ensures that network resources are used efficiently, reducing transmission time and enhancing overall network reliability.

5. Supply Chain Management

In supply chain management, A* is used to optimize routes for delivery vehicles, ensuring timely deliveries while minimizing costs. By calculating the most efficient routes, A* helps logistics companies reduce fuel consumption and improve delivery times.

Route Optimization: A* is used to determine the shortest delivery paths for trucks, minimizing travel distance and reducing operational costs.

Warehouse Navigation: In warehouses, A* helps robots navigate through aisles to pick and transport goods, improving productivity and efficiency.

The algorithm ensures that deliveries are made on time and with minimal expense, making it an essential tool for logistics and supply chain operations.

6. Urban Planning

Urban planners use A* to map out optimal infrastructure routes for new roads, public transportation systems, and utilities. The algorithm helps them determine the most efficient paths while considering various constraints such as terrain, existing infrastructure, and environmental impact.

Road Network Design: A* is applied to optimize the layout of new road networks, ensuring that traffic flows efficiently and that the design meets future demand.

Public Transport Planning: Urban planners also use A* to design bus or subway routes that minimize travel time for passengers while maximizing coverage of the city.

In urban planning, A* helps ensure that cities are designed to be both efficient and sustainable, with optimized traffic flow and minimized environmental impact.

ADVANTAGES AND DISADVANTAGES OF A* SEARCH

ADVANTAGES

1. **Optimal solution:** A* ensures finding the optimal (shortest) path from the start node to the destination node in the weighted graph given an acceptable heuristic function. This optimality is a decisive advantage in many applications where finding the shortest path is essential.
2. **Completeness:** If a solution exists, A* will find it, provided the graph does not have an infinite cost This completeness property ensures that A* can take advantage of a solution if it exists.

3. **Efficiency:** A* is efficient ifan efficient and acceptable heuristic function is used. Heuristics guide the search to a goal by focusing on promising paths and avoiding unnecessary exploration, making A* more efficient than non-aware search algorithms such as breadth-first search or depth-first search.
4. **Versatility:** A* is widely applicable to variousproblem areas, including wayfinding, route planning, robotics, game development, and more. A* can be used to find optimal solutions efficiently as long as a meaningful heuristic can be defined.
5. **Optimized search:** A* maintains a priority order to select the nodes with the minor f(n) value (g(n) and h(n)) for expansion. This allows it to explore promising paths first, which reduces the search space and leads to faster convergence.
6. **Memory efficiency:** Unlike some other search algorithms, such as breadth-first search, A* stores only a limited number of nodes in the priority queue, which makes it memory efficient, especially for large graphs.
7. **Tunable Heuristics:** A*'s performancecan be fine-tuned by selecting different heuristic functions. More educated heuristics can lead to faster convergence and less expanded nodes.
8. **Extensively researched:** A* is a well-established algorithm with decades of research and practical applications. Many optimizations and variations have been developed, making it a reliable and well-understood troubleshooting tool.
9. **Web search:** A* can be used for web-based path search, where the algorithm constantly updates the path according to changes in the environment or the appearance of new It enables real-time decision-making in dynamic scenarios.

DISADVANTAGES

- **Heuristic accuracy:** The performance of the A* algorithm depends heavily on the accuracy of the heuristic function used to estimate the cost from the current node to the If the heuristic is unacceptable (never overestimates the actual cost) or inconsistent (satisfies the triangle inequality), A* may not find an optimal path or may explore more nodes than necessary, affecting its efficiency and accuracy.
- **Memory usage:** A* requires that all visited nodes be kept in memory to keep track of explored paths. Memory usage can sometimes become a significant issue, especially when dealing with an ample search space or limited memory resources.
- **Time complexity:** Although A* is generally efficient, its time complexity can be a concern for vast search spaces or graphs. In the worst case, A* can take exponentially longer to find the optimal path if the heuristic is inappropriate for the problem.
- **Bottleneck at the destination:** In specific scenarios, the A* algorithm needs to explore nodes far from the destination before finally reaching the destination region. This the problem occurs when the heuristic needs to direct the search to the goal early effectively.
- **Cost Binding:** A* faces difficulties when multiple nodes have the same f-value (the sum of the actual cost and the heuristic cost). The strategy used can affect the optimality and efficiency of the discovered path. If not handled correctly, it can lead to unnecessary nodes being explored and slow down the algorithm.
- **Complexity in dynamic environments:** In dynamic environments where the cost of edges or nodes may change during the search, A* may not be suitable because it does not adapt well to such changes. Reformulation from scratch

can be computationally expensive, and D* (Dynamic A*) algorithms were designed to solve this

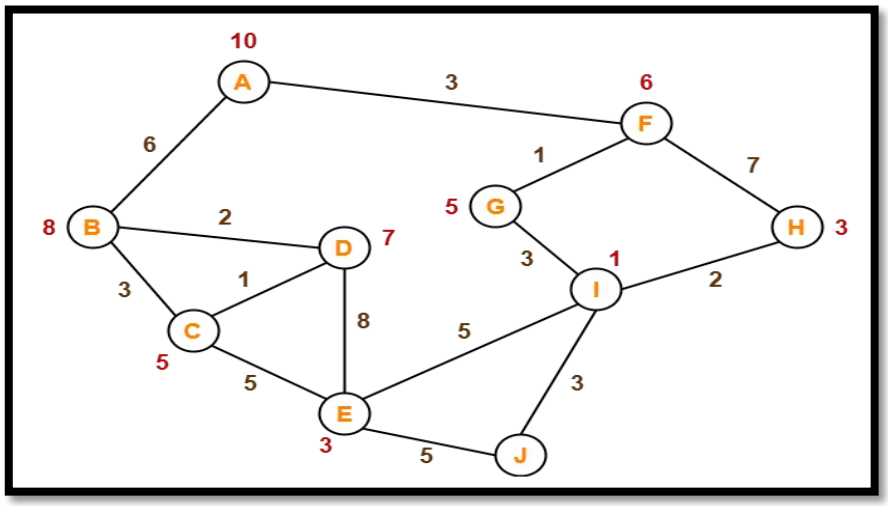
- **Perfection in infinite space :** A* may not find a solution in infinite state space. In such cases, it can run indefinitely, exploring an ever-increasing number of nodes without finding a solution. Despite these shortcomings, A* is still a robust and widely used algorithm because it can effectively find optimal paths in many practical situations if the heuristic function is well-designed and the search space is manageable. Various variations and variants of A* have been proposed to alleviate some of its limitations.
-

Example:

The numbers written on edges represent the distance between the nodes.

The numbers written on nodes represent the heuristic value.

Find the most cost-effective path to reach from start state A to final state J using A* Algorithm.



Solution-

Step-01:

- We start with node A.
- Node B and Node F can be reached from node A.

A* Algorithm calculates f(B) and f(F).

- f(B) = 6 + 8 = 14
- f(F) = 3 + 6 = 9

Since f(F) < f(B), so it decides to go to node F.

Path- A → F

Step-02:

Node G and Node H can be reached from node F.

A* Algorithm calculates f(G) and f(H).

- f(G) = (3+1) + 5 = 9
- f(H) = (3+7) + 3 = 13

Since f(G) < f(H), so it decides to go to node G.

Path- A → F → G

Step-03:

Node I can be reached from node G.

A* Algorithm calculates f(I).

f(I) = (3+1+3) + 1 = 8

It decides to go to node I.

Path- A → F → G → I

Step-04:

Node E, Node H and Node J can be reached from node I.

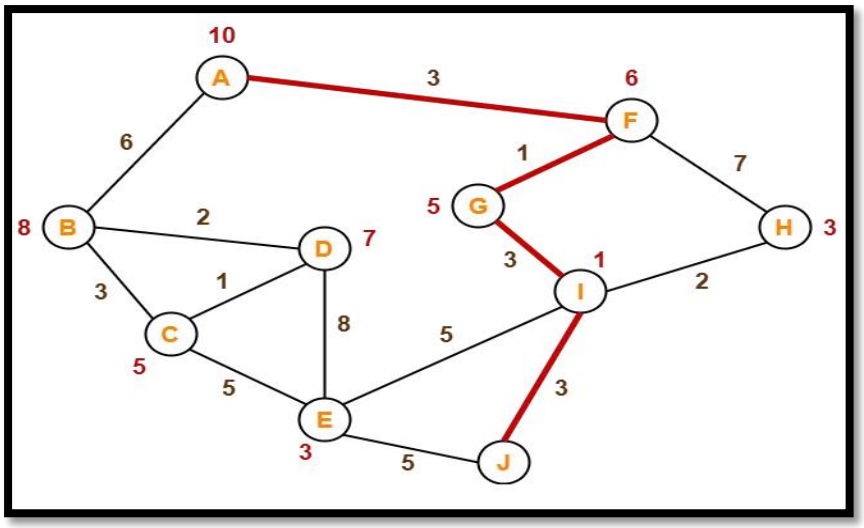
A* Algorithm calculates f(E), f(H) and f(J).

- f(E) = (3+1+3+5) + 3 = 15
- f(H) = (3+1+3+2) + 3 = 12
- f(J) = (3+1+3+3) + 0 = 10

Since f(J) is least, so it decides to go to node J.

Path- A → F → G → I → J

This is the required shortest path from node A to node J.



ITERATIVE DEEPENING A* SEARCH

ITERATIVE DEEPENING A* SEARCH

In terms of memory utilisation, the IDA* algorithm outperforms the A* search algorithm. The whole examined search space is kept in memory by the A* search method, which can be memory-intensive for large search spaces. Contrarily, the IDA* method just saves the current node and its associated cost, not the whole searched area.

In order to explore the search space, the IDA* method employs depth-first search. Starting with a threshold value equal to the heuristic function's anticipated cost from the start node to the destination node. After that, it expands nodes with an overall price less than or equivalent to the threshold value via a depth-first search starting at the start node. The method ends with the best answer if the goal node is located. The algorithm raises the threshold value to the minimal cost of the nodes that were not extended if the threshold value is surpassed. Once the objective node has been located, the algorithm then repeats the procedure.

The IDA* method is full and optimum in the sense that it always finds the best solution if one exists and stops searching if none is discovered. The technique uses less memory since it just saves the current node and its associated cost, not the full search space that has been investigated. Routing, scheduling, and gaming are a few examples of real-world applications where the IDA* method is often employed.

STEPS FOR ITERATIVE DEEPENING A* SEARCH

The IDA* algorithm includes the following steps:

- **Start with an initial cost limit.**

The algorithm begins with an initial cost limit, which is usually set to the heuristic cost estimate of the optimal path to the goal node.

- **Perform a depth-first search (DFS) within the cost limit.**

The algorithm performs a DFS search from the starting node until it reaches a node with a cost that exceeds the current cost limit.

- **Check for the goal node.**

If the goal node is found during the DFS search, the algorithm returns the optimal path to the goal.

- **Update the cost limit.**

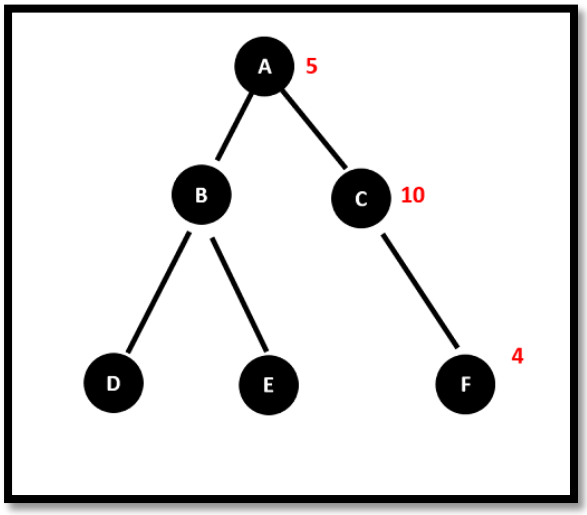
If the goal node is not found during the DFS search, the algorithm updates the cost limit to the minimum cost of any node that was expanded during the search.

- **Repeat the process until the goal is found.**

The algorithm repeats the process, increasing the cost limit each time until the goal node is found.

EXAMPLE OF IDA*

Let's look at a graph example to see how the Iterative Deepening A* (IDA*) technique functions. Assume we have the graph below, where the figures in parenthesis represent the expense of travelling between the nodes:



We want to find the optimal path from node A to node F using the IDA* algorithm. The first step is to set an initial cost limit. Let's use the heuristic estimate of the optimal path, which is 7 (the sum of the costs from A to C to F).

1. Set the cost limit to 7.
2. Start the search at node A.
3. Expand node A and generate its neighbors, B and C.
4. Evaluate the heuristic cost of the paths from A to B and A to C, which are 5 and 10 respectively.

5. Since the cost of the path to B is less than the cost limit, continue the search from node B.
6. Expand node B and generate its neighbors, D and E.
7. Evaluate the heuristic cost of the paths from A to D and A to E, which are 10 and 9 respectively.
8. Since the cost of the path to D exceeds the cost limit, backtrack to node B.
9. Evaluate the heuristic cost of the path from A to C, which is 10.
10. Since the cost of the path to C is less than the cost limit, continue the search from node C.
11. Expand node C and generate its neighbor, F.
12. Evaluate the heuristic cost of the path from A to F, which is 7.
13. Since the cost of the path to F is less than the cost limit, return the optimal path, which is A - C - F.

We're done since the ideal route was discovered within the initial pricing range. We would have adjusted the cost limit to the lowest cost of any node that was enlarged throughout the search and then repeated the procedure until the goal node was located if the best path could not be discovered within the cost limit.

A strong and adaptable search algorithm, the IDA* method may be used to identify the best course of action in a variety of situations. It effectively searches huge state spaces and, if there is an optimal solution, finds it by combining the benefits of DFS and A* search.

ADVANTAGES AND DISADVANTAGES OF IDA*

ADVANTAGES

1. **Completeness:** The IDA* method is a complete search algorithm, which means that, if an optimum solution exists, it will be discovered.
2. **Memory effectiveness:** The IDA* method only keeps one path in memory at a time, making it memory efficient.
3. **Flexibility:** Depending on the application, the IDA* method may be employed with a number of heuristic functions.
4. **Performance:** The IDA* method sometimes outperforms other search algorithms like uniform-cost search (UCS) or breadth-first search (BFS) (UCS).
5. The IDA* algorithm is incremental, which means that it may be stopped at any time and continued at a later time without losing any progress.
6. As long as the heuristic function utilised is acceptable, the IDA* method will always discover the best solution, if one exists. This implies that the algorithm will always choose the route that leads directly to the target node.

DISADVANTAGES

1. Ineffective for huge search areas. IDA* potential for being incredibly ineffective for vast search spaces is one of its biggest drawbacks. Since IDA* expands the same nodes using a depth-first search on each iteration, this might result in repetitive calculations.
2. Get caught in a nearby minima. The ability of IDA* to become caught in local minima is another drawback.

3. Extremely reliant on the effectiveness of the heuristic function. The effectiveness of the heuristic function utilised heavily influences IDA* performance.
4. Although IDA* is memory-efficient in that it only saves one path at a time, there are some situations when it may still be necessary to use a substantial amount of memory.
5. Confined to issues with clear objective states. IDA* is intended for issues when the desired state is clearly defined and identifiable.

EXAMPLE

- **Nodes:** S, A, B, C, G
- **Edges with costs:**
  - S → A (3)
  - S → B (2)
  - A → C (4)
  - B → C (1)
  - C → G (5)
- **Heuristic (h) estimates to G:**
  - h(S) = 5, h(A) = 3, h(B) = 2, h(C) = 1, h(G) = 0

Step-by-Step Execution of IDA*

1. **Initialize**

- Threshold (T) = f(S) = g(S) + h(S) = 0 + 5 = 5
- Stack (DFS frontier): [(S, 0)]

2**. Iteration 1 (T = 5)**

- Expand S:
  - A: f(A) = g(S→A) + h(A) = 3 + 3 = 6 (Exceeds T=5 → Prune)
  - B: f(B) = 2 + 2 = 4 (≤ T → Keep)
- Stack: [(B, 2)]
- Expand B:
  - C: f(C) = (2+1) + 1 = 4 (≤ T → Keep)
- Stack: [(C, 3)]
- Expand C:
  - G: f(G) = (3+5) + 0 = 8 (Exceeds T → Prune)
- Dead end. Update T = min(6, 8) = 6 (next threshold).

3. **Iteration 2 (T = 6)**

- Stack reset: [(S, 0)]
- Expand S:
  - A: f(A) = 6 (≤ T → Keep)
  - B: f(B) = 4 (≤ T → Keep)
- Stack: [(A, 3), (B, 2)] (Order: LIFO)
- Expand A:
  - C: f(C) = (3+4) + 1 = 8 (Exceeds T → Prune)

- Expand B:
  - C: f(C) = 4 (≤ T → Keep)
- Stack: [(C, 3)]
- Expand C:
  - G: f(G) = 8 (Exceeds T → Prune)
- Dead end. Update T = min(8) = 8.

4. **Iteration 3 (T = 8)**

- Stack reset: [(S, 0)]
- Expand S:
  - A: f(A) = 6 (≤ T → Keep)
  - B: f(B) = 4 (≤ T → Keep)
- Stack: [(A, 3), (B, 2)]
- Expand A:
  - C: f(C) = 8 (≤ T → Keep)
- Stack: [(C, 7)]
- Expand C:
  - G: f(G) = (7+5) + 0 = 12 (Exceeds T → Prune)
- Dead end. Update T = min(12) = 12.

5. **Iteration 4 (T = 12)**

- Expand C → G:
  - Path: S → A → C → G
  - Total cost: 3 (S→A) + 4 (A→C) + 5 (C→G) = 12
- Goal found!

Final Answer

- **Optimal Path:** S → A → C → G
- **Cost:** 12

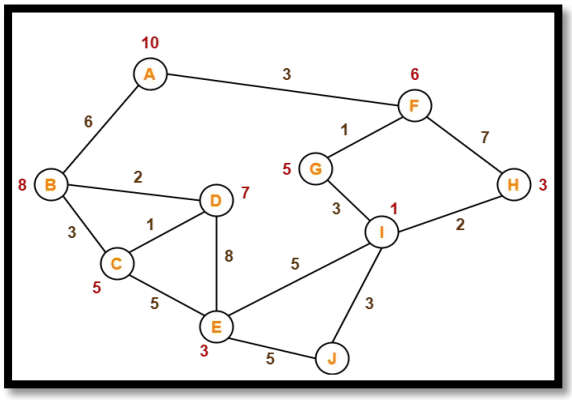
PSEUDOCODE

```
def IDAstar(start, goal):
    T = heuristic(start, goal)  # Initial threshold
    while True:
        min_exceeded = ∞
        stack = [(start, 0)]  # (node, g_cost)
        while stack:
            node, g = stack.pop()
            f = g + heuristic(node, goal)
            if f > T:
                min_exceeded = min(min_exceeded, f)
```

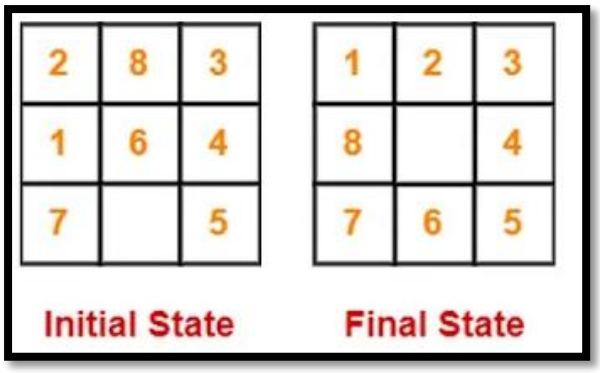
```
                continue
            if node == goal:
                return path
            for neighbor in node.children:
                stack.append((neighbor, g + cost(node,
neighbor)))
        T = min_exceeded  # Update threshold
```

-----QUESTIONS ON UNIT 3-----

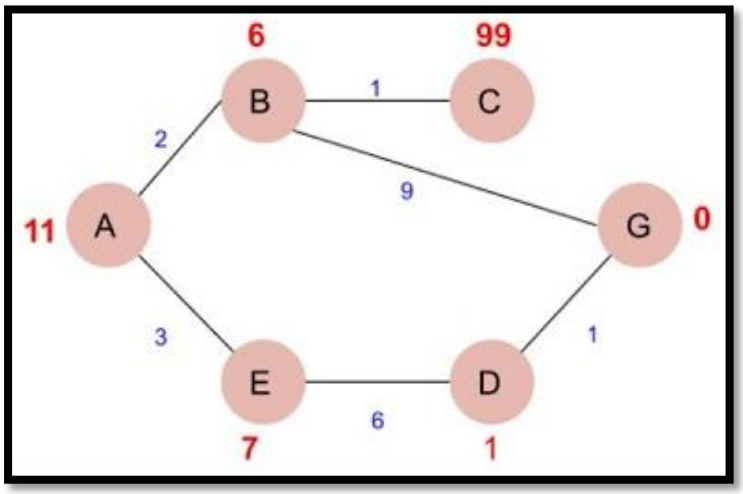
1) Define the term Heuristic Search.
2) Discuss the need to choose the Good Heuristic algorithm.
3) Discuss 8 Puzzle problem with the help of Example.
4) Explain the Monotonicity Modified Travelling salesman Problem with the help of Example. Write its advantages and disadvantages.
5) Write Algorithm for Best First Search.
6) Explain the concept of A* algorithm. Write the algorithmic steps to solve A* algorithm.
7) State the difference between Best First Search and A* Algorithm.
8) Solve the following Example using A* algorithm. (Detail of Each step is expected) Start Node: A Goal Node: J



9) Write the properties of Heuristic Search Algorithm.
10) Define what is Heuristic Function? And where we can use the heuristic function?
11) Explain the Concept of Heuristic Search and explain any 1 type example of Heuristic Search.
12) Solve the following example using 8 Puzzle problem using DFS Approach. Also write algorithm for the same.



13) Solve the following Example using A* algorithm. (Detail of Each step is expected) Start Node: A Goal Node: G



14) Explain Iterative Deepening with A* algorithm, explain in detail with the help of example.

15) Differentiate between A* and Iterative Deeping A* algorithm.

16) Define term Pruning in Iterative Deeping A* algorithm.

17) Write in detail how Iterative Deeping A* algorithm work?

18) Explain the Generalization of the problem in Search algorithm.

19) Define heuristic search and list two advantages it has over uninformed search methods like BFS or DFS.

20) State the Manhattan Distance heuristic formula for the 8-puzzle problem.

21) Define Best Fit Search and list two applications where it is preferred over First Fit Search.

22) State the evaluation function (f(n)) used in the A* algorithm and explain each component.

23) Compare A* and Greedy Best-First Search in terms of heuristic usage and optimality guarantees.

24) Analyze why IDA* is more memory-efficient than A* for large search spaces. Use a search tree example.